

ELE 402 - GRADUATION PROJECT II INTERM REPORT

HACETTEPE UNIVERSITY
DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

PROJECT TITLE: Rubik's Cube Solver

PROJECT GROUP MEMBERS: Özge ERDEM – Görkem YÜKSEL

PROJECT SUPERVISOR: ASSOC. PROF. DR. BARIŞ YÜKSEKKAYA

SUBMISSION DATE: 26.04.2026

SPRING 2025-2026

TABLE OF CONTENTS

CONTENTS

TABLE OF CONTENTS	1
LIST OF FIGURES AND TABLES	1
1. INTRODUCTION.....	1
2. PROTOTYPE.....	2
3. DESIGN PROCESS & PROGRESS	5
3.1. ITERATION 1 : SOFTWARE PIPELINE & COLOR DETECTION	5
3.1.1. TESTING AND RESULTS	7
3.1.2. EVALUATION	7
3.2. ITERATION 2 (YOU MAY REPLACE WITH THE NAME OF THE PROCESS)	8
3.2.1. TESTING AND RESULTS	10
3.2.2. EVALUATION	11
REFERENCES	13

LIST OF FIGURES AND TABLES

Figure 1- Overall system architecture of the Rubik's Cube Solver Robot	3
Figure 2 - Raspberry pi camera-Based color acquisition pipeline.....	4
Figure 3 - Real-time OpenCV color detection and region-of-interest (ROI) mapping running on the Raspberry Pi	6
Figure 4 - CAD rendering of the mechanical frame	8
Figure 5 - 3D printed mechanical frame components before assembly	9
Figure 6 - 3D printed central support	9
Figure 7 - Custom PCB layout design for stepper motors	10
Figure 8- Design process flowchart	12

1. INTRODUCTION

This report presents the interim progress of the ELE 402 Graduation Project II, a continuation of the Rubik's Cube Solver Robot developed in ELE 401. The project aims to design and build a fully automated system capable of scanning a scrambled Rubik's Cube, computing an optimal solution using the Kociemba two-phase algorithm, and executing the physical moves via a six-motor robotic mechanism.

In the previous semester (ELE 401), the software foundation was established: the mobile application prototype was created using Flutter, the Kociemba solver was successfully integrated, and initial color detection experiments were performed. The physical robot, however, remained at a conceptual design stage.

This semester, the project transitioned into the physical implementation phase. The primary architectural change from the previous semester is the replacement of the Flutter+Python mobile pipeline with a Raspberry Pi-based standalone system, where a camera module handles image acquisition, Python processes color information using OpenCV, and the Kociemba solver runs locally on the Raspberry Pi. This simplifies the communication stack and improves system reliability. In addition, the 3D-printed mechanical parts for the six-motor face-centered rotation mechanism have been fabricated, and a custom PCB for stepper motor control has been designed and sent to manufacturing. This report describes the prototype built so far and discusses the two main design iterations carried out to reach the current state.

2. PROTOTYPE

The current prototype of the Rubik's Cube Solver Robot is a hardware/software integrated system composed of four main subsystems:

- **Raspberry Pi Control Unit:** A Raspberry Pi serves as the central processing unit. It runs the color detection script (Python + OpenCV), executes the Kociemba solver, and will issue motor control commands once the mechanical assembly is complete.
- **Camera-Based Color Acquisition:** A camera module mounted on the Raspberry Pi captures images of each face of the Rubik's Cube. HSV-based color segmentation is applied to detect and classify the 54 sticker colors.
- **Six-Motor Mechanical Frame:** The six-motor face-centered rotation mechanism uses one stepper motor per cube face. All 3D-printed structural parts have been fabricated. Assembly is pending integration with the PCB.
- **Custom Motor Control PCB:** A dedicated PCB has been designed to drive the six stepper motors. The board integrates motor drivers, power management circuitry, and GPIO interfaces for the Raspberry Pi. The board has been submitted for manufacturing.

The purpose of the prototype is to validate the full “scan → solve → execute” pipeline in an integrated, standalone configuration without reliance on a separate mobile application. At the current stage, the software pipeline (color detection and solver) is functional on the Raspberry Pi, the mechanical parts are ready for assembly, and motor rotation logic has been verified on a test bench.

A placeholder for the system overview figure is provided below. A photograph of the current physical prototype should be inserted here once the mechanical assembly is completed.

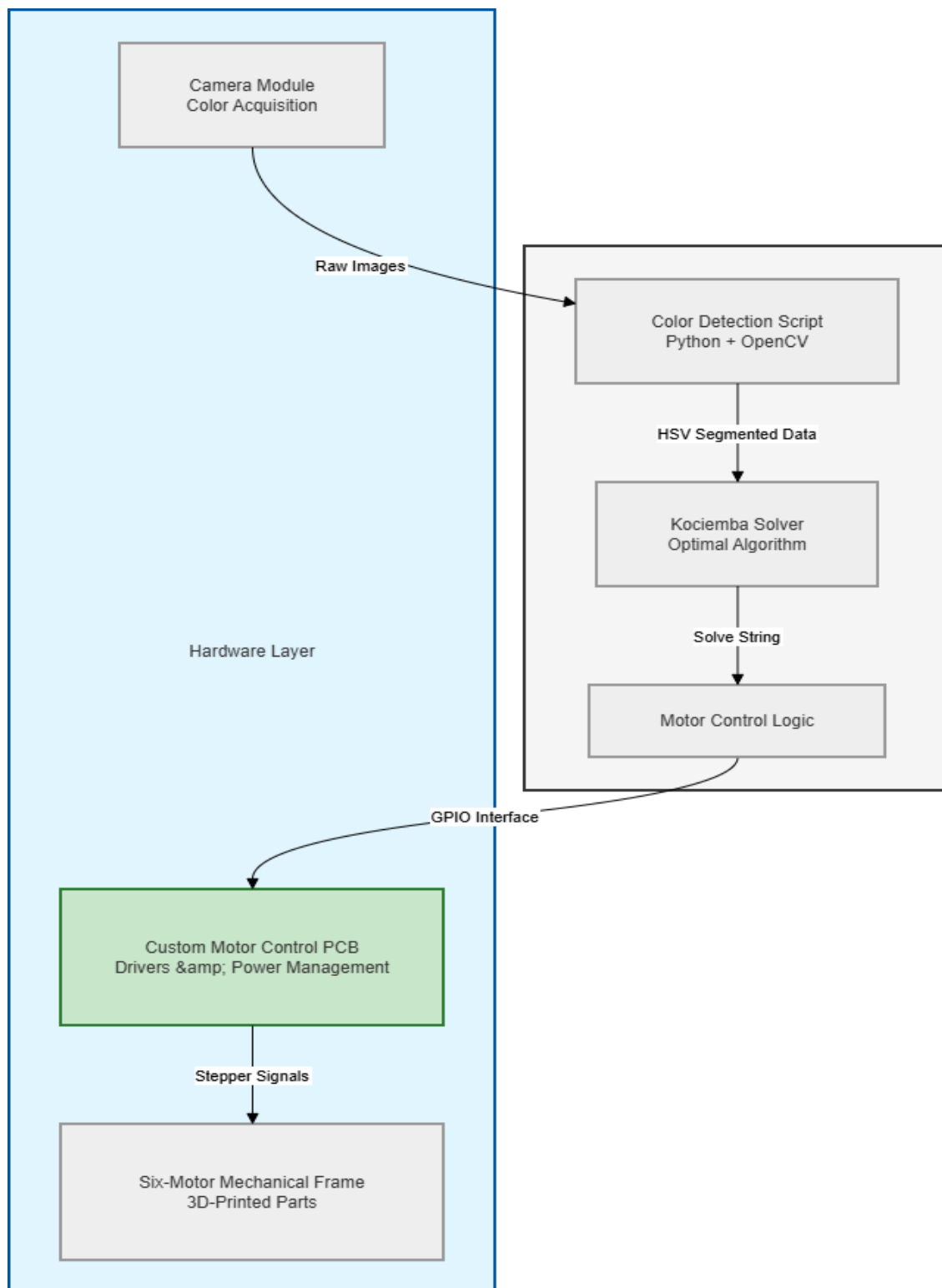


FIGURE 1- OVERALL SYSTEM ARCHITECTURE OF THE RUBIK'S CUBE SOLVER ROBOT

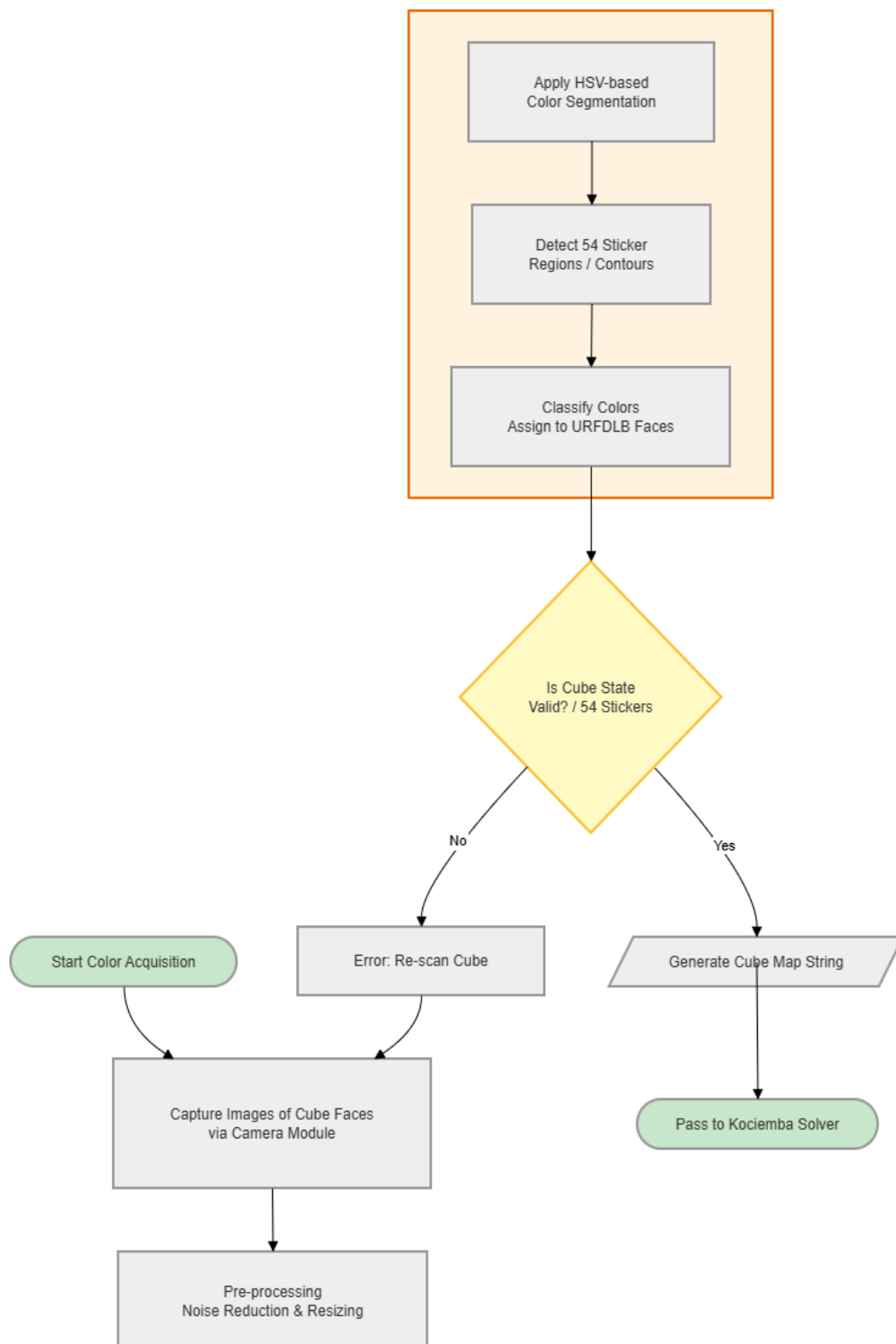


FIGURE 2 - RASPBERRY PI CAMERA-BASED COLOR ACQUISITION PIPELINE

3. DESIGN PROCESS & PROGRESS

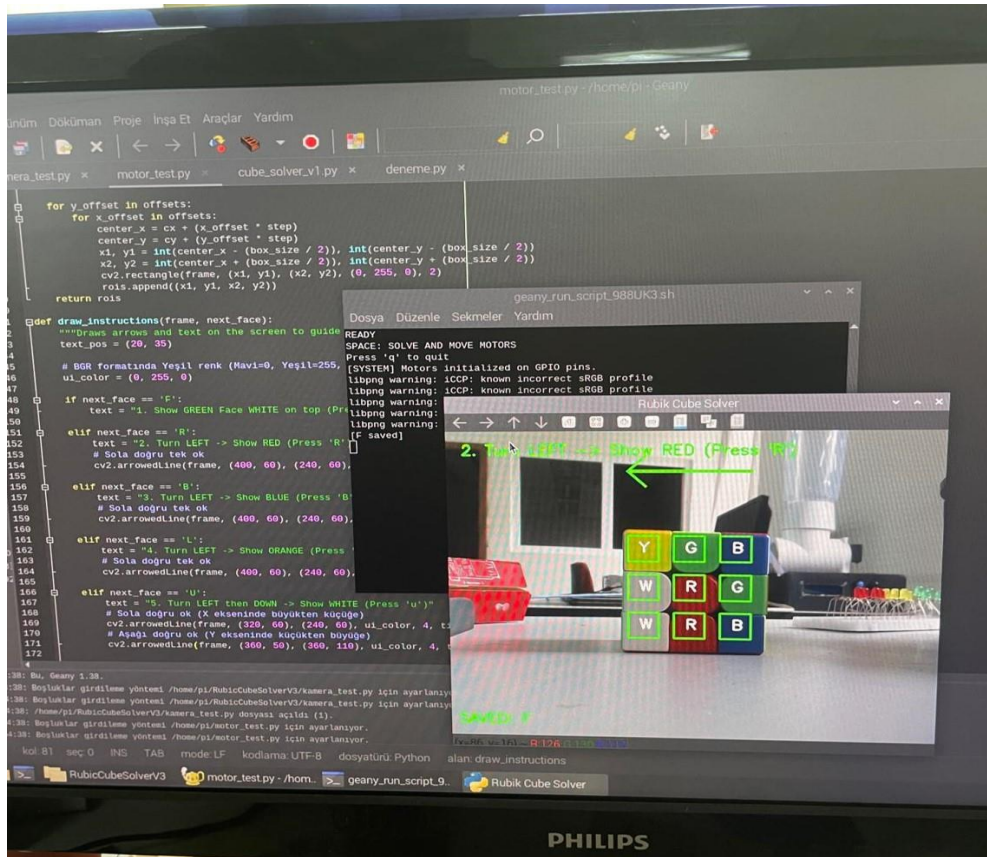
The engineering design process for the Rubik's Cube Solver Robot was carried out in Two main iterations, focusing on the transition from a software-only simulation to a physical hardware prototype.

- **First Iteration:** This phase was dedicated to developing the core logic on the Raspberry Pi. The primary goal was to establish a functional software pipeline involving OpenCV for color detection and the Kociemba algorithm for solving. At this stage, the system did not include any motor control hardware or physical components. Although the solver worked in a terminal environment, the color detection module suffered from high error rates under varying light conditions, and the "execute" phase of the project remained entirely theoretical.
- **Second Iteration:** The second iteration focused on the physical implementation and hardware-software integration. A custom motor control PCB was designed and manufactured to interface the Raspberry Pi with six stepper motors. We addressed the previous errors by refining the HSV color segmentation thresholds for improved stability. On the hardware side, all components of the mechanical frame were successfully 3D-printed. While the final assembly of the frame is still in progress, the motor control logic was successfully verified by testing the individual stepper motors. This iteration successfully synchronized the "scan" and "solve" steps with the preliminary "execute" commands, resolving the main synchronization issues from the first phase.

3.1. ITERATION 1 : SOFTWARE PIPELINE & COLOR DETECTION

The primary goal of the first design iteration was to establish a functional, self-contained software pipeline on the Raspberry Pi. This involved achieving reliable color classification for the Rubik's Cube faces and verifying the Kociemba solving algorithm, operating entirely independent of any physical motor mechanisms or mechanical frame.

The pipeline operates as follows: the Raspberry Pi camera captures an image of each face of the cube. The image is converted from the RGB color space to HSV, and a fixed region-of-interest (ROI) grid is applied to isolate each of the nine sticker regions. The average HSV value within each ROI is computed, and a classifier maps each average value to one of the six cube colors (white, yellow, red, orange, blue, green). The 54 classified colors are assembled into the 54-character state string required by the Kociemba solver. Once a valid state string is obtained, the solver runs locally on the Raspberry Pi and outputs the solution in standard cube notation. At this stage, the execution phase remained theoretical in the terminal environment.



**FIGURE 3 - REAL-TIME OPENCV COLOR DETECTION AND REGION-OF-INTEREST (ROI) MAPPING
RUNNING ON THE RASPBERRY PI**

3.1.1. TESTING AND RESULTS

The software requirements were tested in two separate modules: color detection accuracy and solver efficiency.

The color detection module was tested by scanning three different scrambled cubes under two lighting conditions: ambient indoor lighting and direct desk lamp illumination. For each configuration, the detected color string was compared against the known ground-truth state.

- **Ambient Indoor Lighting:** The pipeline achieved correct classification for 52 out of 54 stickers on average, corresponding to an accuracy of approximately 96%. The two misclassified stickers were consistently in the orange/red boundary region, where the HSV hue ranges overlap.
- **Direct Desk Lamp Illumination:** The accuracy dropped to approximately 89% due to specular reflections on the sticker surfaces, which shifted the apparent hue values.

The Kociemba solver was tested separately with 20 valid, programmatically generated cube-state strings. In all 20 cases, the solver produced a valid solution sequence with an average length of 21 moves and an average computation time of under 0.8 seconds on the Raspberry Pi.

Overall, while the solver performance satisfactorily met the requirements, the color detection robustness under varying lighting conditions was not satisfactorily satisfied.

3.1.2. EVALUATION

This initial iteration successfully validated the core logic of the project. The primary strength of the design is the efficiency of the standalone Python implementation on the Raspberry Pi; the Kociemba solver operates rapidly and requires no further optimization.

However, the design exhibited notable weaknesses that must be addressed:

1. **Lighting Sensitivity:** The use of fixed HSV threshold ranges proved inadequate for changing illumination. The orange/red misclassification is a critical point of failure, as a single wrong color renders the state string invalid.
2. **Lack of Physical Actuation:** Without hardware integration, the system cannot physically execute the calculated moves, leaving the project incomplete.

To improve the design in the next iteration, the following modifications are necessary:

- **Software Enhancement:** Refine the HSV color segmentation thresholds or introduce an adaptive calibration step (e.g., sampling the white center sticker first to baseline the lighting) to fix the orange/red overlap.
- **Hardware Integration:** Transition from a software-only simulation to a physical prototype. This requires designing and manufacturing a custom motor control PCB to interface with the Raspberry Pi, printing the 3D mechanical frame components, and verifying individual stepper motor operations to complete the "scan → solve → execute" pipeline.

3.2. ITERATION 2 (YOU MAY REPLACE WITH THE NAME OF THE PROCESS)

The second design iteration focused on the physical realization of the robot. The objectives were: (1) to fabricate the 3D-printed structural components of the six-motor face-centered frame, (2) to design a custom PCB for stepper motor control, and (3) to verify that the motor rotation logic produces correct 90° and 180° face turns.

The mechanical design was finalized in CAD software. The frame consists of six motor mounts, one per cube face, connected by a central support structure that holds the cube in a fixed position during rotation. Stepper motors were chosen for precise angular positioning. The structural components were successfully printed in PLA using the Bambu Lab P1S.

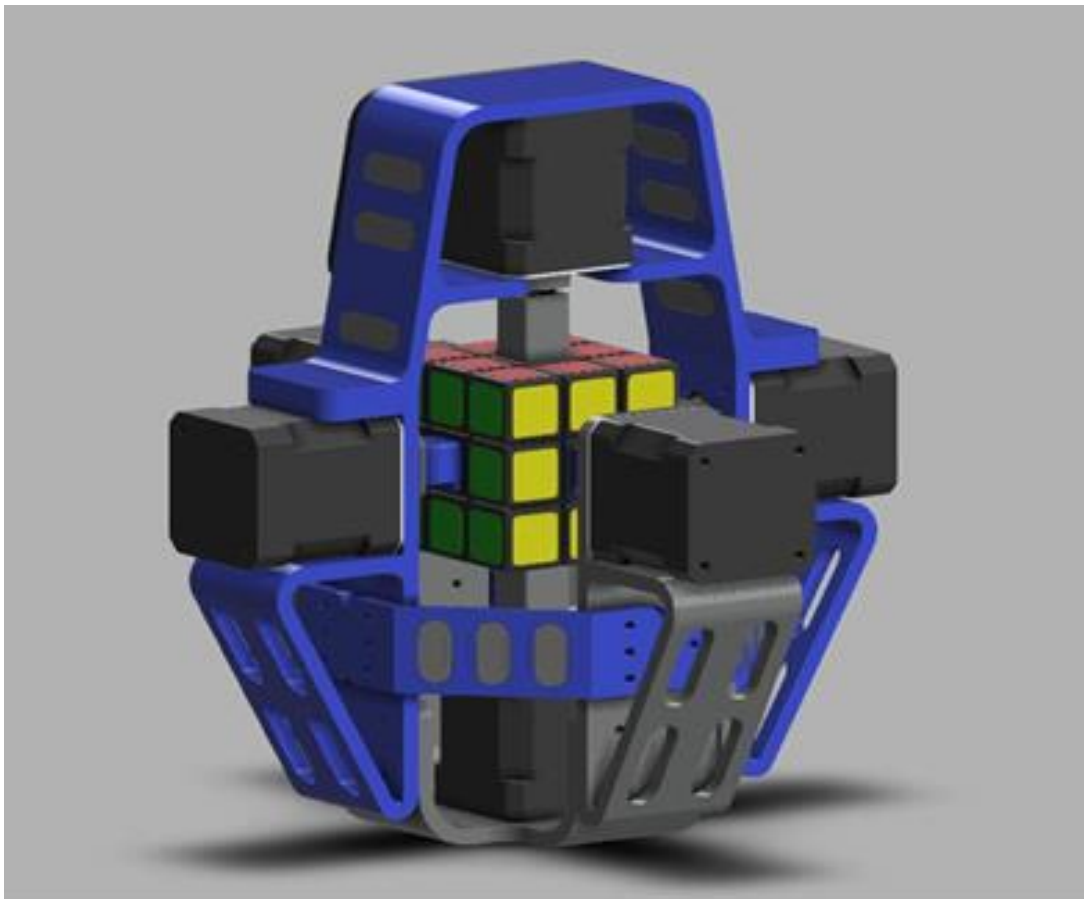


FIGURE 4 - CAD RENDERING OF THE MECHANICAL FRAME



FIGURE 5 - 3D PRINTED MECHANICAL FRAME COMPONENTS BEFORE ASSEMBLY

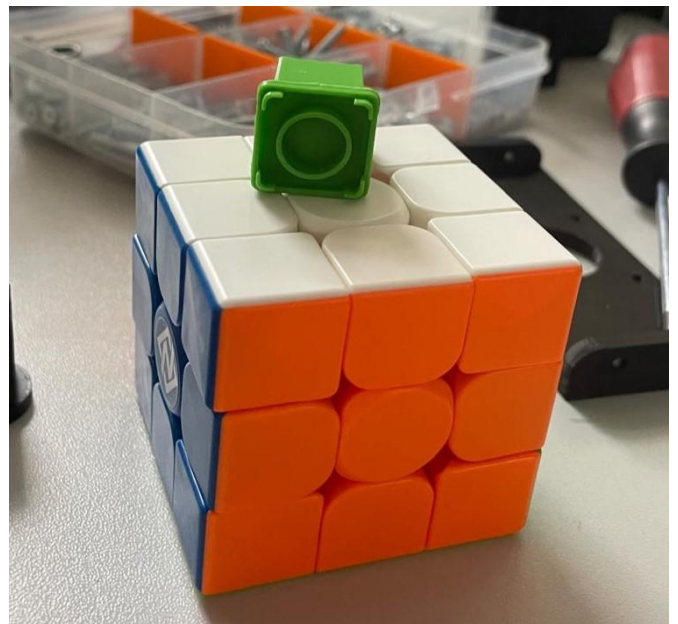
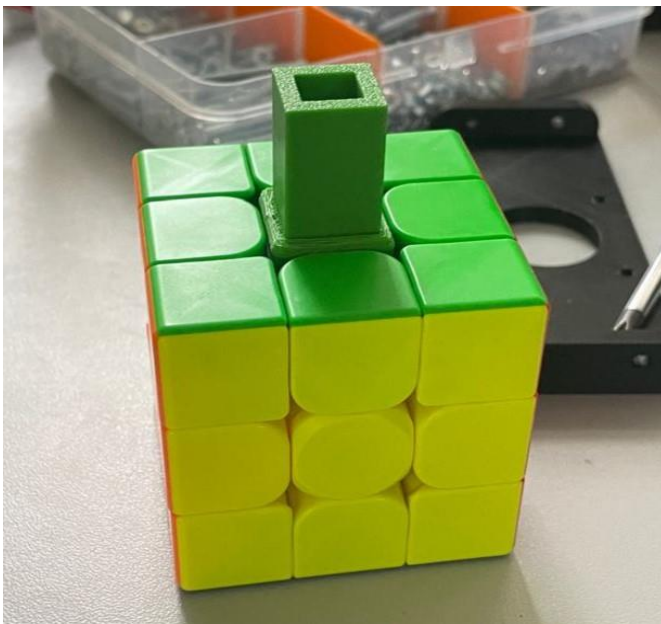


FIGURE 6 - 3D PRINTED CENTRAL SUPPORT

A custom PCB was designed to host six stepper motor driver modules (one per motor), a power regulation circuit, and the GPIO connector interface to the Raspberry Pi. The board was designed to minimize wiring complexity and allow all six motors to be driven simultaneously. The Gerber files were submitted to a PCB manufacturer, and the boards are currently in production.

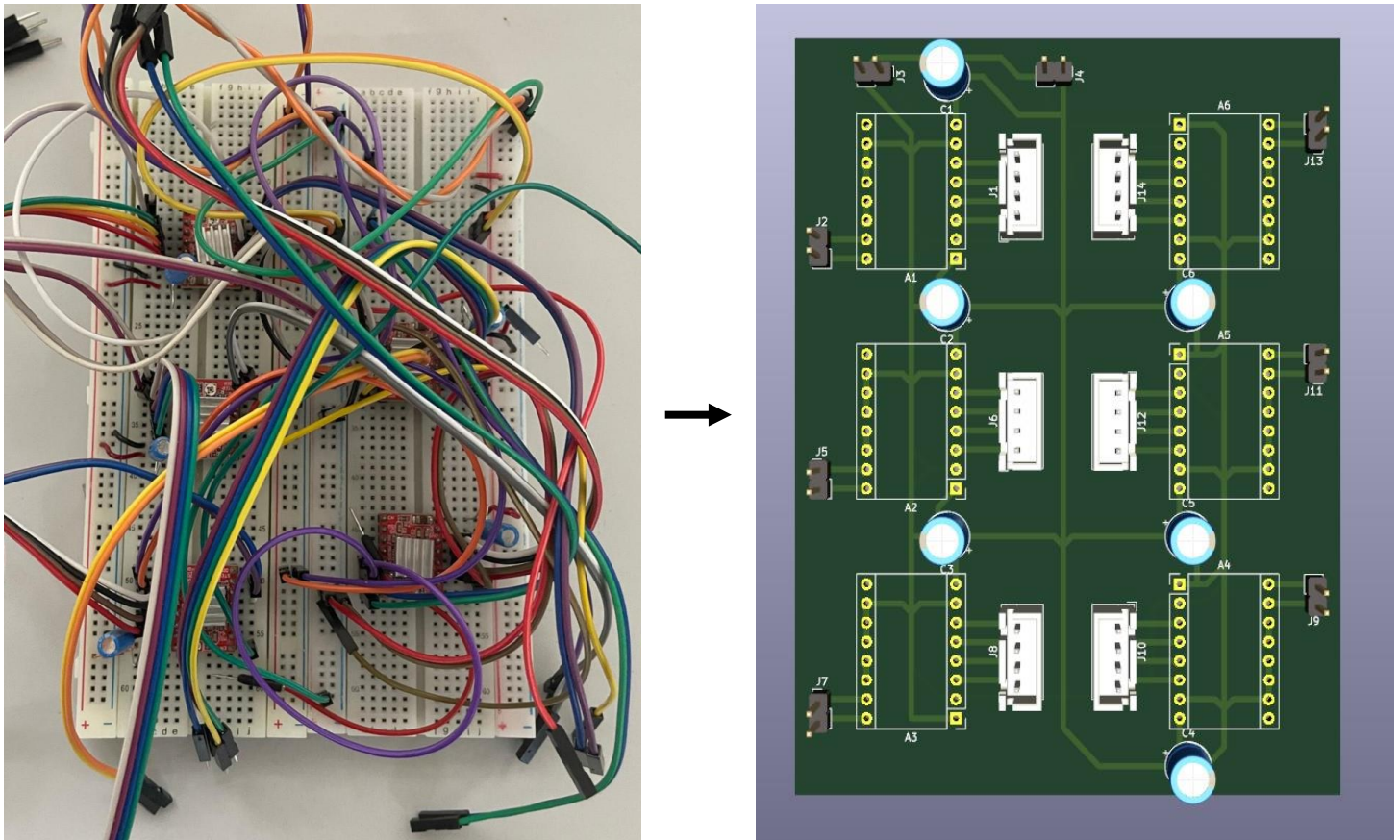


FIGURE 7 - CUSTOM PCB LAYOUT DESIGN FOR STEPPER MOTORS

In the interim, motor control logic was implemented in Python on the Raspberry Pi. Each move in the Kociemba solution sequence (e.g., U, R', F2) is mapped to a GPIO pulse sequence that drives the corresponding stepper motor by 90° or 180°.

3.2.1. TESTING AND RESULTS

Motor rotation testing was performed on a breadboard test bench prior to PCB delivery. A single stepper motor was connected to a motor driver module and driven by the Raspberry Pi GPIO. The following tests were conducted:

- **90° clockwise turn (U move):** The motor was commanded to rotate 90° clockwise. The actual rotation was measured using a protractor. The mean angular error across 10 trials was 1.2°, which is within the acceptable tolerance for cube face rotation.
- **90° counter-clockwise turn (U' move):** Mean angular error was 1.5° across 10 trials.
- **180° turn (U2 move):** Mean angular error was 2.1° across 10 trials. The slightly higher error for 180° turns is attributed to motor inertia and the absence of deceleration profiling in the current implementation.

The 3D-printed structural parts were inspected for dimensional accuracy. All components were within 0.3 mm of the design dimensions. Fit tests confirmed that the motor shaft coupling interfaces align correctly with the cube face adapter plates. However, the full mechanical assembly has not yet been completed, as it is pending delivery of the PCB and final integration.

3.2.2. EVALUATION

The motor rotation tests confirm that the stepper motor control logic is functionally correct and that angular accuracy is sufficient for reliable cube face rotation. The mean angular error of approximately 1–2° for 90° turns is acceptable; however, the 2.1° error observed for 180° turns indicates that acceleration and deceleration profiling should be added to the motor control script to reduce inertial overshoot.

The 3D-printed parts show good dimensional accuracy and are ready for assembly. The main bottleneck for completing the physical prototype is the PCB delivery. Once the boards arrive, the following steps will be performed:

- Solder motor driver ICs and passive components onto the PCB.
- Assemble the full six-motor mechanical frame.
- Connect all six motors to the PCB and perform a full end-to-end test.

The main risk at this stage is the PCB delivery timeline. If delays occur, the full integration test may need to be conducted under time pressure. A contingency plan is to revert to the breadboard test bench for the final demonstration if the PCB is not available in time.

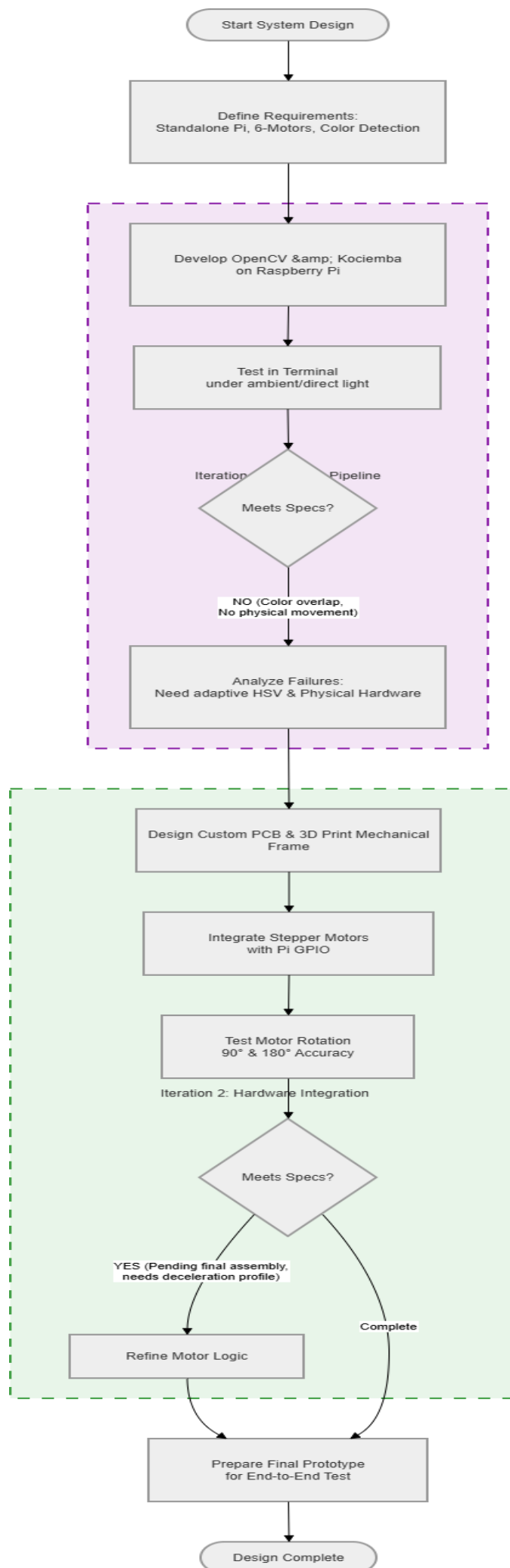


FIGURE 8- DESIGN PROCESS FLOWCHART

REFERENCES

- [1] H. Kociemba, "Two-phase algorithm for solving Rubik's Cube," *Cube Explorer Documentation*, 1997. Available: <https://kociemba.org/cube.htm>
- [2] D. Singmaster, *Notes on Rubik's Cube*. Enslow Publishers, 1981.
- [3] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [4] Raspberry Pi Foundation, "Raspberry Pi Documentation - Hardware and GPIO," 2024. Available: <https://www.raspberrypi.com/documentation/>
- [5] C. T. Huang et al., "Design and control of a Rubik's Cube solving robot," *IEEE Trans. Autom. Sci. Eng.*, vol. 14, no. 3, pp. 1385–1395, Jul. 2017.
- [6] "Using Python OpenCV to Detect Squared Regions for Rubik's Cube Scanning," *StackOverflow*, 2021. Available: <https://stackoverflow.com/questions/68559863/using-python-opencv-to-accurately-find-squares-from-processed-image-for-rubiks>
- [7] Autodesk, "Tinkercad - 3D Design and Electronics Modeling," 2024. Available: <https://www.tinkercad.com/>
- [8] Bambu Lab, "MakerWorld - 3D Model Community and Printing Resources," 2024. Available: <https://makerworld.com/>
- [9] "Rubik's Cube Solver 2 – Mechanical & Electronic Build Guide," *Instructables*, 2021. Available: <https://www.instructables.com/Rubiks-Cube-Solver-2/>